

```
import mysql.connector
import pandas as pd
import matplotlib.pyplot as plt
```

```
mycon = mysql.connector.connect(
    host="localhost",
    user="root",
    password="1234",
    database="EXP_TRACK")
print("MySQL connection sucessfull")
```

MySQL connection sucessfull

```
df = pd.read_sql("SELECT * FROM EXP", mycon)

print(df)

import warnings
warnings.filterwarnings("ignore")
```

|    | EXP_ID | EXP_DATE   | EXP_NAME      | CATEGORY  | AMOUNT   | PAY_METHOD |
|----|--------|------------|---------------|-----------|----------|------------|
| 0  | 1      | 2026-08-02 | LUNCH         | FOOD      | 180.00   | UPI        |
| 1  | 2      | 2026-08-05 | UBER          | TRAVEL    | 156.00   | CASH       |
| 2  | 3      | 2026-08-11 | BOOKS         | EDU       | 288.00   | UPI        |
| 3  | 4      | 2026-08-14 | CLOTHS        | LIFESTYLE | 680.99   | DC         |
| 4  | 5      | 2026-08-14 | EARPHONES     | LIFESTYLE | 1899.89  | CC         |
| 5  | 6      | 2026-08-20 | UBER          | TRAVEL    | 70.53    | UPI        |
| 6  | 7      | 2026-08-21 | SNACKS        | FOOD      | 60.00    | CASH       |
| 7  | 8      | 2026-08-22 | GIFT          | MIS       | 800.00   | UPI        |
| 8  | 9      | 2026-08-26 | ELE_BILL      | BILL      | 2000.00  | UPI        |
| 9  | 10     | 2026-08-27 | LPG           | BILL      | 988.00   | UPI        |
| 10 | 11     | 2026-08-07 | PETROL        | BILL      | 586.00   | UPI        |
| 11 | 12     | 2026-08-01 | GROCERIES     | BILL      | 6000.00  | DC         |
| 12 | 13     | 2026-08-15 | VEGIES        | MIS       | 843.00   | CASH       |
| 13 | 14     | 2026-08-20 | EMI           | BILL      | 15000.00 | NB         |
| 14 | 15     | 2026-08-03 | MEDICINE      | MIS       | 245.00   | UPI        |
| 15 | 16     | 2026-08-04 | MOVIE         | MIS       | 420.00   | CC         |
| 16 | 17     | 2026-08-06 | COFFEE        | FOOD      | 140.00   | CASH       |
| 17 | 18     | 2026-08-08 | METRO         | TRAVEL    | 85.00    | UPI        |
| 18 | 19     | 2026-08-09 | PEN_DRIVE     | LIFESTYLE | 650.00   | DC         |
| 19 | 20     | 2026-08-10 | COURSE        | EDU       | 1200.00  | UPI        |
| 20 | 21     | 2026-08-12 | DINNER        | FOOD      | 560.00   | CC         |
| 21 | 22     | 2026-08-13 | SHOES         | LIFESTYLE | 1499.00  | DC         |
| 22 | 23     | 2026-08-16 | MEDICINE      | MIS       | 380.00   | CASH       |
| 23 | 24     | 2026-08-17 | NETFLIX       | BILL      | 649.00   | UPI        |
| 24 | 25     | 2026-08-18 | PHONE_CASE    | LIFESTYLE | 499.00   | UPI        |
| 25 | 26     | 2026-08-19 | AUTO          | TRAVEL    | 120.00   | CASH       |
| 26 | 27     | 2026-08-23 | BIRTHDAY_GIFT | MIS       | 950.00   | CC         |
| 27 | 28     | 2026-08-24 | BREAKFAST     | FOOD      | 210.00   | UPI        |
| 28 | 29     | 2026-08-25 | ONLINE_BOOK   | EDU       | 375.00   | NB         |
| 29 | 30     | 2026-08-28 | BAG           | LIFESTYLE | 2299.00  | DC         |
| 30 | 31     | 2026-09-01 | BREAKFAST     | FOOD      | 160.00   | UPI        |
| 31 | 32     | 2026-09-02 | UBER          | TRAVEL    | 210.00   | CASH       |
| 32 | 33     | 2026-09-04 | NOTEBOOKS     | EDU       | 340.00   | UPI        |
| 33 | 34     | 2026-09-06 | TSHIRT        | LIFESTYLE | 799.00   | DC         |
| 34 | 35     | 2026-09-08 | ELECTRICITY   | BILL      | 1750.00  | UPI        |
| 35 | 36     | 2026-09-10 | SNACKS        | FOOD      | 95.00    | CASH       |
| 36 | 37     | 2026-09-12 | PIZZA         | FOOD      | 620.00   | CC         |
| 37 | 38     | 2026-09-14 | ONLINE_COURSE | EDU       | 999.00   | NB         |
| 38 | 39     | 2026-09-16 | BAG           | LIFESTYLE | 1250.00  | DC         |
| 39 | 40     | 2026-09-17 | MOBILE_BILL   | BILL      | 699.00   | UPI        |

C:\Users\Admin\AppData\Local\Temp\ipykernel\_11764\1436807951.py:1: UserWarning: pandas only supports SQLAlchemy connectable  
df = pd.read\_sql("SELECT \* FROM EXP", mycon)

```
from datetime import datetime

#cd=current date, cm=current month, current year.
cd = datetime.now()
cm = cd.month
cy = cd.year

print("Current Month:", cm)
print("Current Year:", cy)
```

Current Month: 9  
Current Year: 2026

```
query = " SELECT * FROM EXP WHERE MONTH(EXP_DATE) = %s AND YEAR(EXP_DATE) = %s "

current_df = pd.read_sql(query, mycon, params=(cm, cy))
```

current\_df

|   | EXP_ID | EXP_DATE   | EXP_NAME      | CATEGORY  | AMOUNT | PAY_METHOD |
|---|--------|------------|---------------|-----------|--------|------------|
| 0 | 31     | 2026-09-01 | BREAKFAST     | FOOD      | 160.0  | UPI        |
| 1 | 32     | 2026-09-02 | UBER          | TRAVEL    | 210.0  | CASH       |
| 2 | 33     | 2026-09-04 | NOTEBOOKS     | EDU       | 340.0  | UPI        |
| 3 | 34     | 2026-09-06 | TSHIRT        | LIFESTYLE | 799.0  | DC         |
| 4 | 35     | 2026-09-08 | ELECTRICITY   | BILL      | 1750.0 | UPI        |
| 5 | 36     | 2026-09-10 | SNACKS        | FOOD      | 95.0   | CASH       |
| 6 | 37     | 2026-09-12 | PIZZA         | FOOD      | 620.0  | CC         |
| 7 | 38     | 2026-09-14 | ONLINE_COURSE | EDU       | 999.0  | NB         |
| 8 | 39     | 2026-09-16 | BAG           | LIFESTYLE | 1250.0 | DC         |
| 9 | 40     | 2026-09-17 | MOBILE_BILL   | BILL      | 699.0  | UPI        |

```
choice = int(input("Enter 1 for current month or 2 to select another month: "))

# sm=selected month, sy=selected year.
if choice == 1:
    sm = cm
    sy = cy
elif choice == 2:
    sm = int(input("Enter month (1-12): "))
    sy = int(input("Enter year: "))
else:
    print("Invalid choice")
```

Enter 1 for current month or 2 to select another month: 1

```
query = "SELECT * FROM EXP WHERE MONTH(EXP_DATE) = %s AND YEAR(EXP_DATE) = %s"

selected_df = pd.read_sql(query, mycon, params=(sm, sy))

selected_df
```

|   | EXP_ID | EXP_DATE   | EXP_NAME      | CATEGORY  | AMOUNT | PAY_METHOD |
|---|--------|------------|---------------|-----------|--------|------------|
| 0 | 31     | 2026-09-01 | BREAKFAST     | FOOD      | 160.0  | UPI        |
| 1 | 32     | 2026-09-02 | UBER          | TRAVEL    | 210.0  | CASH       |
| 2 | 33     | 2026-09-04 | NOTEBOOKS     | EDU       | 340.0  | UPI        |
| 3 | 34     | 2026-09-06 | TSHIRT        | LIFESTYLE | 799.0  | DC         |
| 4 | 35     | 2026-09-08 | ELECTRICITY   | BILL      | 1750.0 | UPI        |
| 5 | 36     | 2026-09-10 | SNACKS        | FOOD      | 95.0   | CASH       |
| 6 | 37     | 2026-09-12 | PIZZA         | FOOD      | 620.0  | CC         |
| 7 | 38     | 2026-09-14 | ONLINE_COURSE | EDU       | 999.0  | NB         |
| 8 | 39     | 2026-09-16 | BAG           | LIFESTYLE | 1250.0 | DC         |
| 9 | 40     | 2026-09-17 | MOBILE_BILL   | BILL      | 699.0  | UPI        |

```
print("Add a new EXP")

qry = "SELECT MAX(EXP_ID) FROM EXP"

id_df = pd.read_sql(qry, mycon)

if pd.isna(id_df.iloc[0, 0]):
    new_id = 1
else:
    new_id = int(id_df.iloc[0, 0]) + 1

print("New Expense ID:", new_id)

exp_date = input("Enter expense date (YYYY-MM-DD): ")
exp_name = input("Enter expense name: ")
category = input("Enter category: ").upper()
amount = float(input("Enter amount: "))
pay_method = input("Enter payment method: ").upper()
```

```

qry = """INSERT INTO EXP
        VALUES (%s, %s, %s, %s, %s, %s)"""

val = (new_id, exp_date, exp_name, category, amount, pay_method)

mycursor = mycon.cursor()
mycursor.execute(qry, val)
mycon.commit()

print("Expense added successfully!")

# Display the newly added expense
new_exp_df = pd.read_sql(
    "SELECT * FROM EXP WHERE EXP_ID = %s",
    mycon,
    params=(new_id,)
)

print("NEW EXPENSE")
print(new_exp_df)

```

```

Add a new EXP
New Expense ID: 41
Enter expense date (YYYY-MM-DD): 2026-09-11
Enter expense name: test 1
Enter category: mis
Enter amount: 589.25
Enter payment method: upi
Expense added successfully!
NEW EXPENSE

```

| EXP_ID | EXP_DATE | EXP_NAME   | CATEGORY | AMOUNT | PAY_METHOD |     |
|--------|----------|------------|----------|--------|------------|-----|
| 0      | 41       | 2026-09-11 | test 1   | MIS    | 589.25     | UPI |

```

print("SEARCH EXPENSES")
print("-----")

# Load complete expense table
search_df = pd.read_sql(
    "SELECT * FROM EXP", mycon)

print("1. Search by Expense ID")
print("2. Search by Date Range")
print("3. Search by Expense Name")
print("4. Search by Category")
print("5. Search by Amount")
print("6. Search by Payment Method")

result = search_df.copy()

while True:

    sc = int(input("Enter search type: "))

    if sc == 1:
        EXP_ID = int(input("Enter EXP_ID: "))
        result = result[result["EXP_ID"] == EXP_ID]

    elif sc == 2:
        start_date = input("Enter starting date (YYYY-MM-DD): ")
        end_date = input("Enter ending date (YYYY-MM-DD): ")

        start_date = pd.to_datetime(start_date)
        end_date = pd.to_datetime(end_date)

        result["EXP_DATE"] = pd.to_datetime(result["EXP_DATE"])

        result = result[
            (result["EXP_DATE"] >= start_date) &
            (result["EXP_DATE"] <= end_date)]

    elif sc == 3:
        name = input("Enter expense name: ").upper()
        result = result[result["EXP_NAME"] == name]

    elif sc == 4:
        category = input("Enter category: ").upper()
        result = result[result["CATEGORY"] == category]

    elif sc == 5:
        print("1. Amount greater than")
        print("2. Amount less than")
        print("3. Amount equal to")

```

```

ac = int(input("Enter amount condition: "))
amount = float(input("Enter amount: "))

if ac == 1:
    result = result[result["AMOUNT"] > amount]

elif ac == 2:
    result = result[result["AMOUNT"] < amount]

elif ac == 3:
    result = result[result["AMOUNT"] == amount]

else:
    print("Invalid amount condition")

elif sc == 6:
    pay_method = input("Enter payment method: ").upper()
    result = result[result["PAY_METHOD"] == pay_method]

else:
    print("Invalid search type")

print("\nCURRENT SEARCH RESULT")
print("-----")
print(result)

more = input("\nDo you want to add another search condition? (Y/N): ").upper()

if more == "N":
    break
elif more != "Y":
    print("Invalid choice")
    break

print("\nFINAL SEARCH RESULT")
print("-----")
print(result)

```

#### SEARCH EXPENSES

-----

1. Search by Expense ID
2. Search by Date Range
3. Search by Expense Name
4. Search by Category
5. Search by Amount
6. Search by Payment Method

Enter search type: 2

Enter starting date (YYYY-MM-DD): 2026-08-20

Enter ending date (YYYY-MM-DD): 2026-09-20

#### CURRENT SEARCH RESULT

-----

|    | EXP_ID | EXP_DATE   | EXP_NAME      | CATEGORY  | AMOUNT   | PAY_METHOD |
|----|--------|------------|---------------|-----------|----------|------------|
| 5  | 6      | 2026-08-20 | UBER          | TRAVEL    | 70.53    | UPI        |
| 6  | 7      | 2026-08-21 | SNACKS        | FOOD      | 60.00    | CASH       |
| 7  | 8      | 2026-08-22 | GIFT          | MIS       | 800.00   | UPI        |
| 8  | 9      | 2026-08-26 | ELE_BILL      | BILL      | 2000.00  | UPI        |
| 9  | 10     | 2026-08-27 | LPG           | BILL      | 988.00   | UPI        |
| 13 | 14     | 2026-08-20 | EMI           | BILL      | 15000.00 | NB         |
| 26 | 27     | 2026-08-23 | BIRTHDAY_GIFT | MIS       | 950.00   | CC         |
| 27 | 28     | 2026-08-24 | BREAKFAST     | FOOD      | 210.00   | UPI        |
| 28 | 29     | 2026-08-25 | ONLINE_BOOK   | EDU       | 375.00   | NB         |
| 29 | 30     | 2026-08-28 | BAG           | LIFESTYLE | 2299.00  | DC         |
| 30 | 31     | 2026-09-01 | BREAKFAST     | FOOD      | 160.00   | UPI        |
| 31 | 32     | 2026-09-02 | UBER          | TRAVEL    | 210.00   | CASH       |
| 32 | 33     | 2026-09-04 | NOTEBOOKS     | EDU       | 340.00   | UPI        |
| 33 | 34     | 2026-09-06 | TSHIRT        | LIFESTYLE | 799.00   | DC         |
| 34 | 35     | 2026-09-08 | ELECTRICITY   | BILL      | 1750.00  | UPI        |
| 35 | 36     | 2026-09-10 | SNACKS        | FOOD      | 95.00    | CASH       |
| 36 | 37     | 2026-09-12 | PIZZA         | FOOD      | 620.00   | CC         |
| 37 | 38     | 2026-09-14 | ONLINE_COURSE | EDU       | 999.00   | NB         |
| 38 | 39     | 2026-09-16 | BAG           | LIFESTYLE | 1250.00  | DC         |
| 39 | 40     | 2026-09-17 | MOBILE_BILL   | BILL      | 699.00   | UPI        |

Do you want to add another search condition? (Y/N): y

Enter search type: 6

Enter payment method: upi

#### CURRENT SEARCH RESULT

-----

|    | EXP_ID | EXP_DATE   | EXP_NAME  | CATEGORY | AMOUNT  | PAY_METHOD |
|----|--------|------------|-----------|----------|---------|------------|
| 5  | 6      | 2026-08-20 | UBER      | TRAVEL   | 70.53   | UPI        |
| 7  | 8      | 2026-08-22 | GIFT      | MIS      | 800.00  | UPI        |
| 8  | 9      | 2026-08-26 | ELE_BILL  | BILL     | 2000.00 | UPI        |
| 9  | 10     | 2026-08-27 | LPG       | BILL     | 988.00  | UPI        |
| 27 | 28     | 2026-08-24 | BREAKFAST | FOOD     | 210.00  | UPI        |

|    |    |            |             |      |         |     |
|----|----|------------|-------------|------|---------|-----|
| 30 | 31 | 2026-09-01 | BREAKFAST   | FOOD | 160.00  | UPI |
| 32 | 33 | 2026-09-04 | NOTEBOOKS   | EDU  | 340.00  | UPI |
| 34 | 35 | 2026-09-08 | ELECTRICITY | BILL | 1750.00 | UPI |
| 39 | 40 | 2026-09-17 | MOBILE_BILL | BILL | 699.00  | UPI |

Do you want to add another search condition? (Y/N): y  
Enter search type: 4  
Enter category: BILL

```
EXP_ID = int(input("Enter EXP_ID to update: "))

# Find the expense directly using EXP_ID
check = pd.read_sql(
    "SELECT * FROM EXP WHERE EXP_ID = %s", mycon, params=(EXP_ID,))

if check.empty:
    print("Expense not found.")
else:
    print("EXPENSE TO BE UPDATED")
    print(check)

    print("1. Update expense name")
    print("2. Update category")
    print("3. Update amount")
    print("4. Update payment method")

    uc = int(input("Enter your choice: "))

    if uc == 1:
        new_value = input("Enter new expense name: ").upper()
        qry = "UPDATE EXP SET EXP_NAME = %s WHERE EXP_ID = %s"
        val = (new_value, EXP_ID)

    elif uc == 2:
        new_value = input("Enter new category: ").upper()
        qry = "UPDATE EXP SET CATEGORY = %s WHERE EXP_ID = %s"
        val = (new_value, EXP_ID)

    elif uc == 3:
        new_value = float(input("Enter new amount: "))
        qry = "UPDATE EXP SET AMOUNT = %s WHERE EXP_ID = %s"
        val = (new_value, EXP_ID)

    elif uc == 4:
        new_value = input("Enter payment method: ").upper()
        qry = "UPDATE EXP SET PAY_METHOD = %s WHERE EXP_ID = %s"
        val = (new_value, EXP_ID)

    else:
        print("Invalid choice")
        qry = None

    if qry is not None:
        mycursor = mycon.cursor()
        mycursor.execute(qry, val)
        mycon.commit()

        print("Expense updated successfully!")

        # Display the updated record
        updated_df = pd.read_sql(
            "SELECT * FROM EXP WHERE EXP_ID = %s", mycon, params=(EXP_ID,))

        print("UPDATED EXPENSE")
        print(updated_df)
```

```
Enter EXP_ID to update: 41
EXPENSE TO BE UPDATED
  EXP_ID  EXP_DATE EXP_NAME CATEGORY  AMOUNT PAY_METHOD
0     41  2026-09-11  test 1      MIS   589.25        UPI
1. Update expense name
2. Update category
3. Update amount
4. Update payment method
Enter your choice: 2
Enter new category: food
Expense updated successfully!
UPDATED EXPENSE
  EXP_ID  EXP_DATE EXP_NAME CATEGORY  AMOUNT PAY_METHOD
0     41  2026-09-11  test 1      FOOD   589.25        UPI
```

```
EXP_ID = int(input("Enter EXP_ID to delete: "))
```

```

# Find the expense directly using EXP_ID
check = pd.read_sql(
    "SELECT * FROM EXP WHERE EXP_ID = %s", mycon, params=(EXP_ID,))

if check.empty:
    print("Expense not found.")
else:
    print("EXPENSE TO BE DELETED")
    print(check)

    confirm = input("Do you want to delete this expense? (Y/N): ").upper()

    if confirm == "Y":

        qry = "DELETE FROM EXP WHERE EXP_ID = %s"
        val = (EXP_ID,)

        mycursor = mycon.cursor()
        mycursor.execute(qry, val)
        mycon.commit()

        print("Expense deleted successfully!")

    elif confirm == "N":
        print("Deletion cancelled.")

    else:
        print("Invalid choice")

```

```

Enter EXP_ID to delete: 41
EXPENSE TO BE DELETED
  EXP_ID  EXP_DATE EXP_NAME CATEGORY  AMOUNT PAY_METHOD
0      41  2026-09-11   test 1     FOOD   589.25        UPI
Do you want to delete this expense? (Y/N): y
Expense deleted successfully!

```

```

print("EXPENSE ANALYSIS")
print("-----")

if selected_df.empty:
    print("No expenses found for the selected month.")

else:
    print("1. Expense Analysis")
    print("2. Category-wise Expenditure")
    print("3. Payment Method-wise Expenditure")
    print("4. Daily Expenditure")
    print("5. Number of Expenses by Category")
    print("6. Monthly Analysis Summary")

    ac = int(input("Enter your choice: "))

    if ac == 1:
        total_exp = selected_df["AMOUNT"].sum()
        highest_exp = selected_df["AMOUNT"].max()
        lowest_exp = selected_df["AMOUNT"].min()
        avg_exp = selected_df["AMOUNT"].mean()
        total_records = selected_df["EXP_ID"].count()

        print("\nEXPENSE ANALYSIS")
        print("-----")
        print("Total Expenditure:", total_exp)
        print("Highest Expenditure:", highest_exp)
        print("Lowest Expenditure:", lowest_exp)
        print("Average Expenditure:", round(avg_exp, 2))
        print("Number of Expenses:", total_records)

    elif ac == 2:
        category_exp = selected_df.groupby("CATEGORY")["AMOUNT"].sum()

        print("\nCATEGORY-WISE EXPENDITURE")
        print("-----")
        print(category_exp)

    elif ac == 3:
        payment_exp = selected_df.groupby("PAY_METHOD")["AMOUNT"].sum()

        print("\nPAYMENT METHOD-WISE EXPENDITURE")
        print("-----")
        print(payment_exp)

    elif ac == 4:

```

```

daily_exp = selected_df.groupby("EXP_DATE")["AMOUNT"].sum()

print("\nDAILY EXPENDITURE")
print("-----")
print(daily_exp)

highest_day = daily_exp.idxmax()
highest_day_exp = daily_exp.max()

print("\nHighest Spending Day:", highest_day)
print("Amount Spent on That Day:", highest_day_exp)

elif ac == 5:
    category_count = selected_df.groupby("CATEGORY")["EXP_ID"].count()

    print("\nNUMBER OF EXPENSES BY CATEGORY")
    print("-----")
    print(category_count)

elif ac == 6:
    total_exp = selected_df["AMOUNT"].sum()
    highest_exp = selected_df["AMOUNT"].max()
    lowest_exp = selected_df["AMOUNT"].min()
    avg_exp = selected_df["AMOUNT"].mean()
    total_records = selected_df["EXP_ID"].count()

    analysis = {
        "Total Expenditure": total_exp,
        "Highest Expenditure": highest_exp,
        "Lowest Expenditure": lowest_exp,
        "Average Expenditure": round(avg_exp, 2),
        "Number of Expenses": total_records
    }

    analysis_df = pd.DataFrame(
        analysis.items(),
        columns=["ANALYSIS", "VALUE"]
    )

    print("\nMONTHLY ANALYSIS SUMMARY")
    print("-----")
    print(analysis_df)

else:
    print("Invalid choice")

```

```

EXPENSE ANALYSIS
-----
1. Expense Analysis
2. Category-wise Expenditure
3. Payment Method-wise Expenditure
4. Daily Expenditure
5. Number of Expenses by Category
6. Monthly Analysis Summary
Enter your choice: 5

NUMBER OF EXPENSES BY CATEGORY
-----
CATEGORY
BILL      2
EDU       2
FOOD      3
LIFESTYLE  2
TRAVEL    1
Name: EXP_ID, dtype: int64

```

```

print("EXPENSE VISUALISATION")
print("-----")

if selected_df.empty:
    print("No expenses found for the selected month.")
else:
    print("1. Bar Graph")
    print("2. Histogram")

    vc = int(input("Enter your choice: "))

    if vc == 1:

        print("\nBAR GRAPH")
        print("-----")

        # EXP ID is excluded because it is only an identifier

```

```

graph_columns = [
    "EXP_DATE",
    "EXP_NAME",
    "CATEGORY",
    "AMOUNT",
    "PAY_METHOD"
]

print("Available parameters:")

for i, column in enumerate(graph_columns, 1):
    print(i, ".", column)

x_choice = int(input("Select parameter for X-axis: "))
y_choice = int(input("Select parameter for Y-axis: "))

x_column = graph_columns[x_choice - 1]
y_column = graph_columns[y_choice - 1]

plt.figure(figsize=(10, 5))

plt.bar(
    selected_df[x_column].astype(str),
    selected_df[y_column]
)

plt.xlabel(x_column)
plt.ylabel(y_column)
plt.title(y_column + " by " + x_column)

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()

elif vc == 2:

    print("\nHISTOGRAM")
    print("-----")

    # Only meaningful numerical parameter for histogram
    numeric_columns = ["AMOUNT"]

    print("Available numerical parameters:")

    for i, column in enumerate(numeric_columns, 1):
        print(i, ".", column)

    h_choice = int(input("Select parameter: "))

    h_column = numeric_columns[h_choice - 1]

    plt.figure(figsize=(10, 5))

    plt.hist(
        selected_df[h_column],
        bins=10,
        edgecolor="black"
    )

    plt.xlabel(h_column)
    plt.ylabel("Frequency")
    plt.title("Distribution of " + h_column)

    plt.tight_layout()
    plt.show()

else:
    print("Invalid choice")

```

EXPENSE VISUALISATION

- 1. Bar Graph  
2. Histogram  
Enter your choice: 2

HISTOGRAM

-----  
Available numerical parameters:  
1 . AMOUNT  
Select parameter: 1

